

Week 6: Threat Modeling

From information-security reasoning to software controls

Risk → Requirement

Threat → Design

Control → Verify

Conceptual foundation before implementation labs

(AI-Generated Slides)

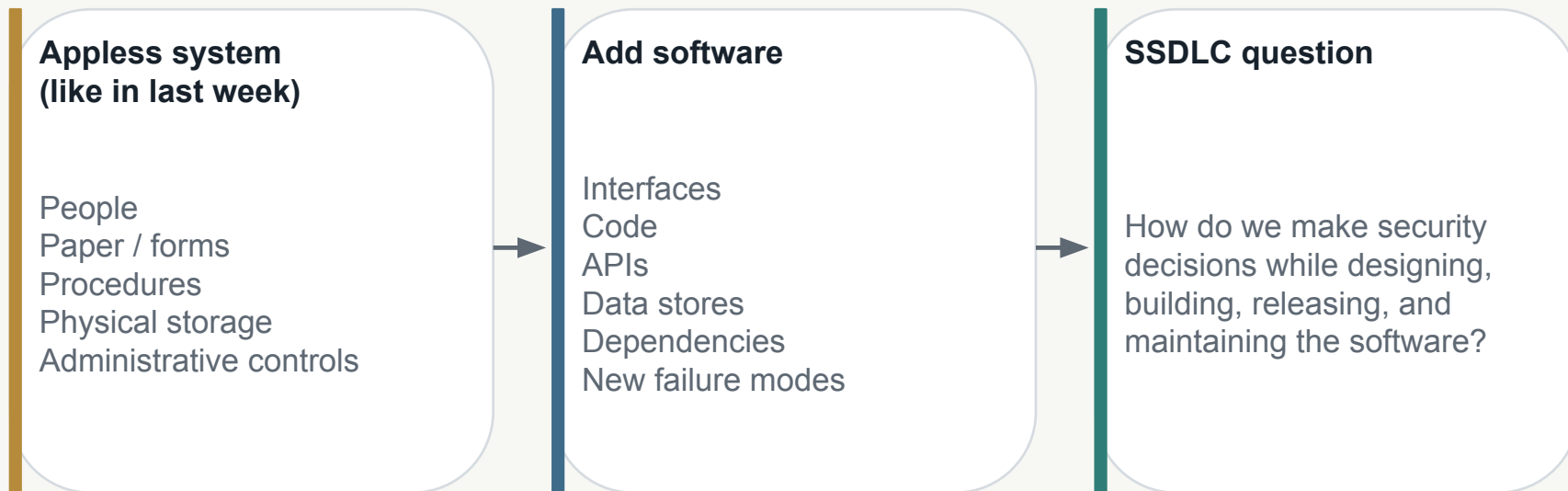
Learning outcomes

By the end of this unit, you should be able to reason from a system to defensible security controls.

- Explain where security decisions occur across the software lifecycle.
- Model a simple system using actors, components, data flows, data stores, and trust boundaries.
- Distinguish threat, vulnerability, risk, control, and residual risk.
- Turn credible threat scenarios into testable security requirements.
- Choose security controls relevant to the work and the threats.

Security existed before the app

Software changes the system; it does not create the need for information security.

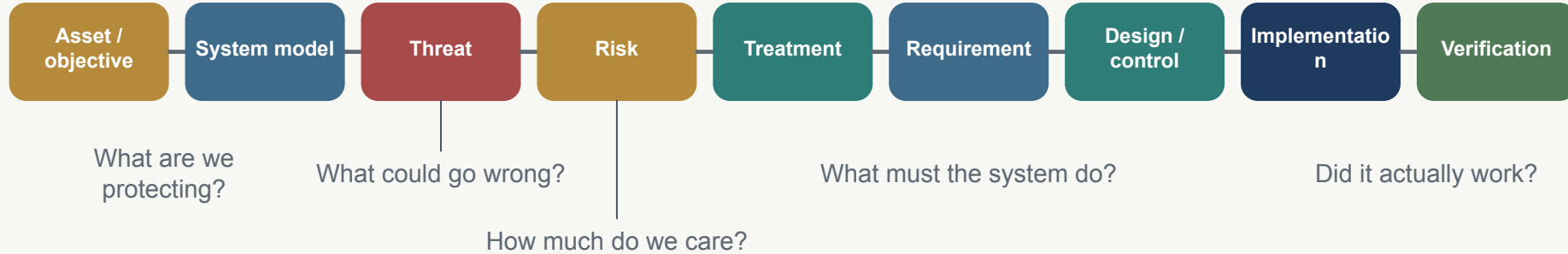


The protection objective may stay the same while threats, probabilities, and available controls change.

NIST SSDF: secure development practices are integrated into an organization's chosen SDLC implementation.

The backbone of this unit

Keep returning to this chain.



Residual risk + feedback loops back into the next iteration

Risk assessment ≠ threat modeling

They interact, but they answer different questions.

Threat modeling

Discover and describe credible ways the system could fail, be misused, or be attacked.

Core question: WHAT COULD GO WRONG?

Output: threat / abuse scenarios and candidate mitigations.

Risk assessment

Evaluate the significance of a scenario in context.

Course model:
 $\text{PROBABILITY} \times \text{EXPOSURE} \rightarrow \text{RISK}$

Output: prioritization and treatment decision.

Threat gives us the bad story. Risk tells us how much that story matters.

OWASP Threat Modeling uses "What can go wrong?" as a core question; risk terminology here uses the course's probability × exposure model.

Threats are not just “hackers”

Model undesirable outcomes—not only villainous intent.

Malicious outsider

credential theft,
exploitation, fraud

Malicious insider

abuse of
legitimate access

Compromised insider

legitimate identity
controlled by
attacker

Mistaken user

wrong target,
wrong scope,
misunderstanding

System / process failure

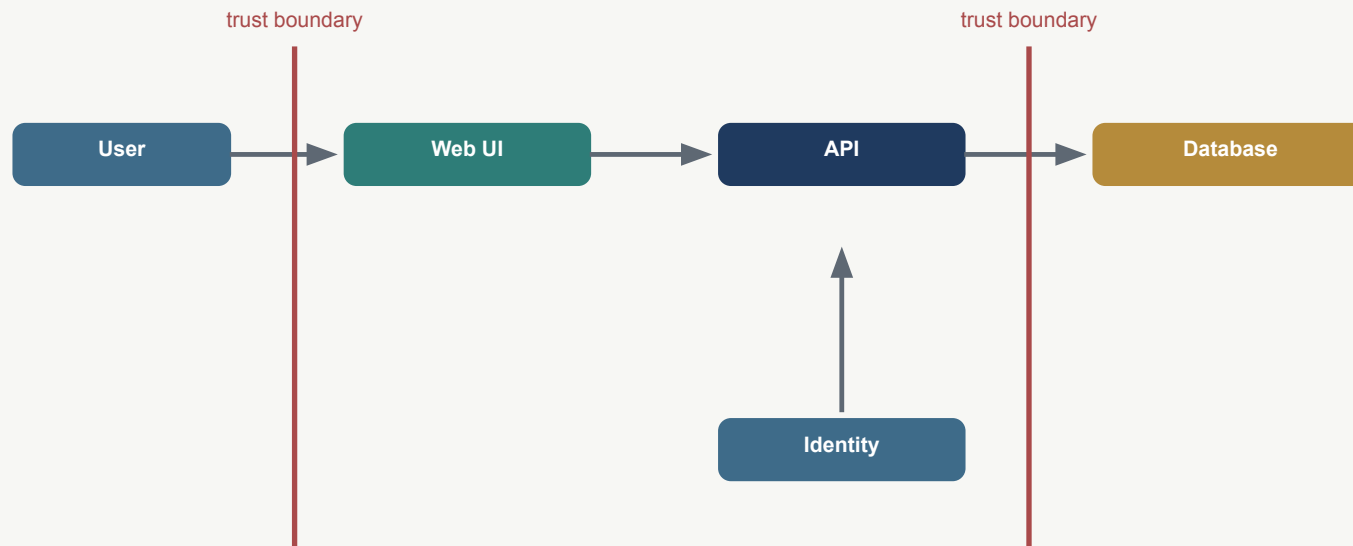
bad defaults,
race, outage,
dependency
failure

A security objective can fail even when nobody intended harm.

OWASP Threat Modeling explicitly includes threats, misuse cases, design assumptions, and security/privacy concerns.

Model the system before modeling threats

You cannot reason well about a system you have not described.



- Actors
- Components
- Data flows
- Data stores
- Entry points
- Trust boundaries

OWASP Threat Modeling: first understand what you are working on, including system, users, dependencies, assumptions, and trust boundaries.

What is a trust boundary?

A **trust boundary** can be thought of as a line drawn through a program. On one side of the line, data is untrusted. On the other side of the line, data is assumed to be trustworthy.

The purpose of **validation logic** is to allow data to safely cross the trust boundary - to move from untrusted to trusted.

A **trust boundary violation** occurs when a program blurs the line between what is trusted and what is untrusted. By combining trusted and untrusted data in the same data structure, it becomes easier for programmers to mistakenly trust unvalidated data.

— [CWE-501](#)

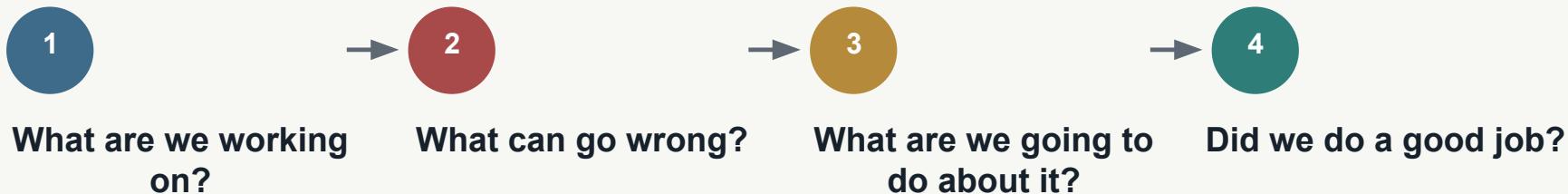


What are some trust boundaries in the real world?

(This slide is not AI-generated. The image is (gpt-image-2 + ChatGPT 5.6 Astra Medium), but not the whole slide.)

Threat modeling: four questions

A methodology-neutral loop before choosing a technique such as STRIDE.



STRIDE is a prompting technique for question 2—not the definition of threat modeling.

Source: OWASP Threat Modeling Project, which presents Shostack's Four Question Framework as a methodology-neutral starting point.

STRIDE as prompts, not a vocabulary test

Use categories to ask better questions about components and boundaries.

S — Spoofing

Can someone pretend to be another identity?

T — Tampering

Can data or code be modified improperly?

R — Repudiation

Can an action become difficult to attribute or prove?

I — Information disclosure

Can information reach someone who should not receive it?

D — Denial of service

Can legitimate use be made unavailable?

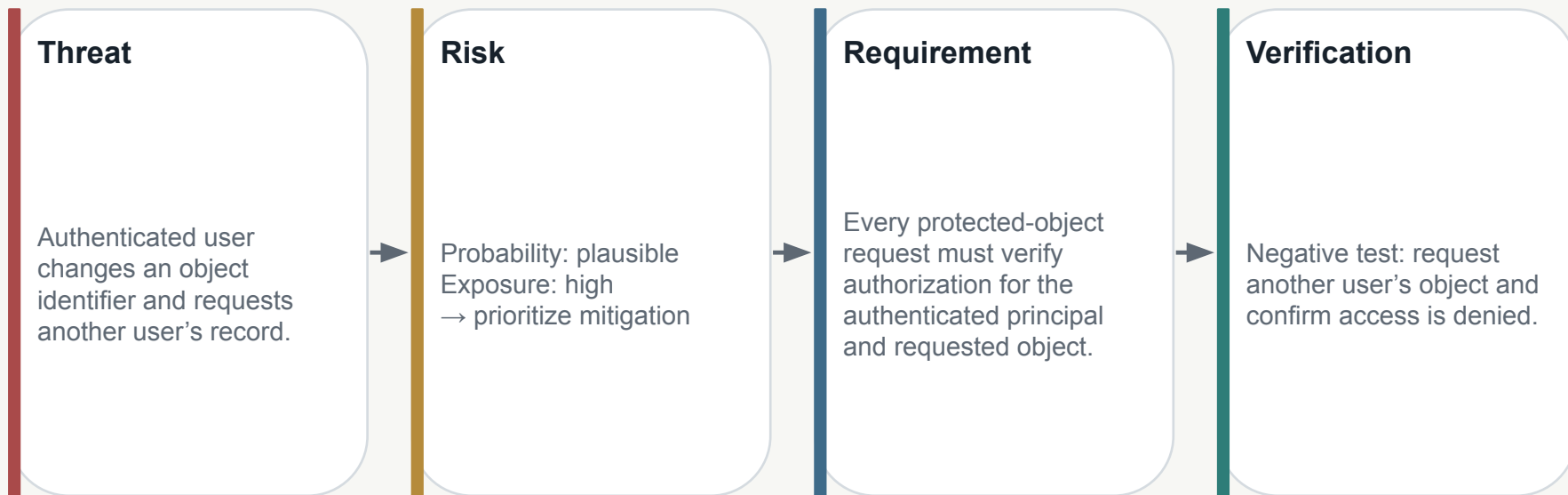
E — Elevation of privilege

Can someone gain capabilities they should not have?

Threat-modeling source: OWASP Threat Modeling Project. STRIDE is one of several recognized approaches; teach it as a structured prompt.

From threat to requirement

The analysis is useful only when it changes what we build or verify.



OWASP ASVS provides requirements for secure development and a basis for testing technical security controls; OWASP Authorization Cheat Sheet recommends validating permissions on every request.

Secure design: reduce probability and bound exposure

A control can prevent the event—or reduce the blast radius when prevention fails.

Prevent

- Least privilege
- Deny by default
- Validate authorization
- Secure defaults
- Input constraints

Bound

- LIMIT / cardinality guardrails
- Rate / quota limits
- Scoped credentials
- Segmentation
- Small transaction boundaries

Recover

- Reversibility
- Backups
- Audit trail
- Rollback
- Incident response

Design for the mistake you know will eventually happen.

Authorization principles supported by OWASP Authorization Cheat Sheet; "bound exposure" is the course design framing.

Destructive operations: authorization is not enough

The operator may be fully authorized—and still make a catastrophic mistake.



- Single-object destructive operation: uniquely identify one intended object; fail closed if that invariant is not met.
- Bulk operation: show scope/count before execution; require deliberate confirmation.
- High-blast-radius action: consider stronger separation, reversibility, or two-person control.
- Guardrail principle: do not depend on “the operator will be careful.”

LIMIT 1 is a guardrail—not proof that the correct row was selected.

Strategic friction

Sometimes good security UX intentionally makes a dangerous action harder.

Routine + reversible → Minimal friction

Sensitive → Explicit confirmation

High impact → Preview + deliberate friction

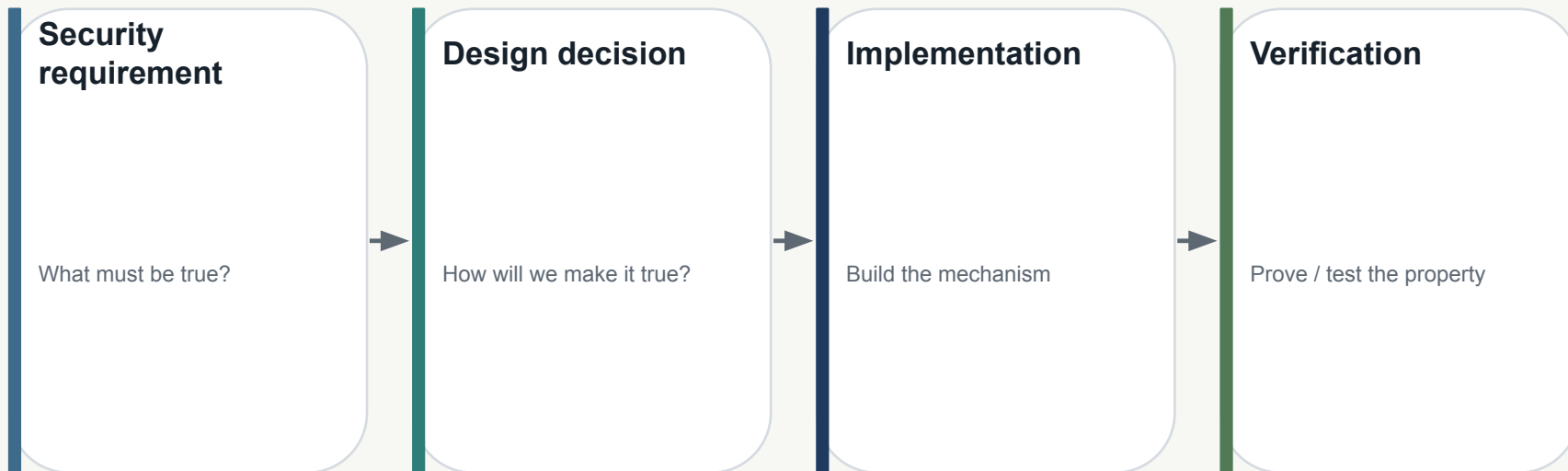
Catastrophic / irreversible → Strong safeguards / separation

An OTP-shaped interaction may be used for authentication, reauthentication, transaction authorization—or simply deliberate friction.

Ask: what security property is this control intended to provide?

Implementation comes later—but requirements come first

The implementation lab should be the consequence of earlier analysis.



**Later implementation topics: authorization • input validation • sessions • secrets • cryptography
• logging • APIs • dependencies**

NIST SSDF is outcome-oriented; OWASP ASVS can supply concrete security requirements and verification targets.

Verification is not “run a scanner”

Different mechanisms answer different questions.

Review

Does the design/code implement the intended requirement?

SAST

What suspicious code patterns can static analysis find?

SCA

What known dependency / component risks are present?

DAST

What issues are observable through a running interface?

Security tests

Do specific security requirements hold?

Pen test

What attack paths can skilled adversarial testing uncover?

NIST SSDF includes producing well-secured software and responding to vulnerabilities; OWASP ASVS provides a basis for testing application technical security controls.

Finding ≠ vulnerability ≠ risk

Context still matters after testing.



A scanner severity is evidence—not automatically your business risk decision.

- Validate the finding.
- Understand exploit conditions and affected assets.
- Assess probability and exposure in context.
- Treat, accept, escalate, or redesign.
- Reassess residual risk after controls.

Capstone exercise

Given a small architecture, produce the security reasoning—not just a list of bugs.

1. Assets & objectives

2. Simple DFD / trust boundaries

3. Threat scenarios

4. Probability × exposure

5. Treatment decisions

6. Testable requirements

7. Proposed controls

8. Verification plan

Success criterion: another team should be able to implement and test your requirements without guessing what “secure” means.

References

Primary sources used in this deck.

NIST SP 800-218 (2022)

Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities

<https://csrc.nist.gov/pubs/sp/800/218/final>

OWASP Threat Modeling Project

Maintained guidance; Four Question Framework; multiple threat-modeling methodologies

<https://owasp.org/www-project-threat-modeling/>

OWASP ASVS 5.0.0

Application Security Verification Standard; requirements for secure development and basis for testing controls

<https://owasp.org/www-project-application-security-verification-standard/>

OWASP Authorization Cheat Sheet

Least privilege, deny by default, validate permissions on every request, test authorization logic

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

Note: NIST SP 800-218 Rev. 1 / SSDF v1.2 was still an Initial Public Draft in the source check; this deck therefore anchors normative claims to final v1.1.